APACHE AIRFLOW + SPARK

1. PROJECT OVERVIEW

This guide covers the Python/PySpark part of building an end-to-end data engineering project using Apache Airflow, Docker, and a Spark cluster. The project demonstrates how to orchestrate Spark jobs written in Python using Airflow's SparkSubmitOperator.

Focus: Creating a PySpark job (e.g., WordCount) and submitting it to a Spark cluster via Airflow DAG running in Docker.

2. ARCHITECTURE OVERVIEW

The setup uses Docker Compose to spin up:

- Spark Master + Workers
- Airflow Webserver + Scheduler (based on Python 3.12 image)
- PostgreSQL as Airflow metadata database
- Shared volumes for jobs/ directory between containers

Airflow DAG uses SparkSubmitOperator to submit the Python job to the Spark master (spark://spark-master:7077).

3. TECHNOLOGY STACK

Component	Technology	Role in Python Job Flow
Orchestration	Apache Airflow 2.10	DAG definition, scheduling, SparkSubmitOperator
Compute	Apache Spark (Standalone)	PySpark job execution on cluster
Language	Python 3.12+ PySpark	Spark job code (wordcountjob.py)
Containerization	Docker + Docker Compose	Unified environment for Airflow + Spark
Storage	Shared Docker Volume (jobs/)	Job scripts accessible to both Airflow & Spark
Metadata DB	PostgreSQL	Airflow task history and state

4. CREATING THE PYSPARK JOB

4.1 Job Location & Structure

In the SparkingFlow project, Python jobs are placed in: jobs/python/

- Example file: jobs/python/wordcountjob.py

4.2 PySpark WordCount Job

A typical PySpark job used in the tutorial:

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("WordCountJob") \
    .getOrCreate()

# Sample text
text = "Hello Spark Hello Python Hello Airflow Hello Docker Hello Himanshu"

# Simple word count using RDD
words = spark.sparkContext.parallelize(text.split(" "))
word_counts = words.map(lambda word: (word, 1)) \
                   .reduceByKey(lambda a :{__add__}, b: a + b)

# Print results
print("=== Word Count Results ===")
for word, count in sorted(word_counts.collect()):
    print(f"{word}: {count}")

spark.stop()
```

5. SUBMITTING THE PYTHON JOB VIA AIRFLOW

5.1 DAG Structure (sparking_flow)

The main DAG typically includes:

- **start task**: PythonOperator that prints a start message
- **python_job task**: SparkSubmitOperator pointing to the Python job
- **end task**: PythonOperator that prints an end message
- Dependencies: start >> python_job >> end

5.2 SparkSubmitOperator Configuration

Key parameters used in the DAG for the Python job:

- task_id: 'python_job'
- conn_id: configured Spark connection (spark-conn)
- application: 'jobs/python/wordcountjob.py' (path inside container)
- master: 'spark://spark-master:7077'
- deploy_mode: 'client'
- executor_cores, executor_memory as needed

Important: The application path must be relative to the volume mount shared between Airflow and Spark containers.

6. KEY TECHNICAL POINTS

- Use shared Docker volumes so Airflow can see the job files and Spark can execute them.
- Ensure Java is installed in the Airflow image (required for spark-submit even for PySpark).
- Set proper SPARK_HOME and PATH in the Airflow container if needed.

- Use env_vars in SparkSubmitOperator if job needs environment variables.
- For production, consider packaging dependencies or using virtualenvs with --py-files.
- Test the job manually first with spark-submit before wiring it into Airflow.

7. COMMON ISSUES & TROUBLESHOOTING

- JAVA_HOME not set → Install OpenJDK in Airflow Dockerfile and set ENV JAVA_HOME.
- File not found → Check volume mounts in docker-compose.yml (jobs/ → /opt/spark/jobs).
- Connection error → Verify spark://master:7077 is reachable from Airflow container (use same network).
- Permission issues → Ensure the airflow user has read access to job files.

8. SKILLS DEMONSTRATED

- Writing and packaging simple PySpark applications (DataFrame API)
- Integrating PySpark jobs with Apache Airflow using SparkSubmitOperator
- Docker Compose multi-service architecture (Airflow + Spark Master/Workers)
- Volume mounting strategies for job code sharing
- Basic orchestration patterns (start → submit job)
- Understanding spark-submit mechanics from Airflow context

9. LIVE EXECUTION

Below are screenshots from actual running SparkingFlow environment, confirming successful implementation of the Python job submission flow.

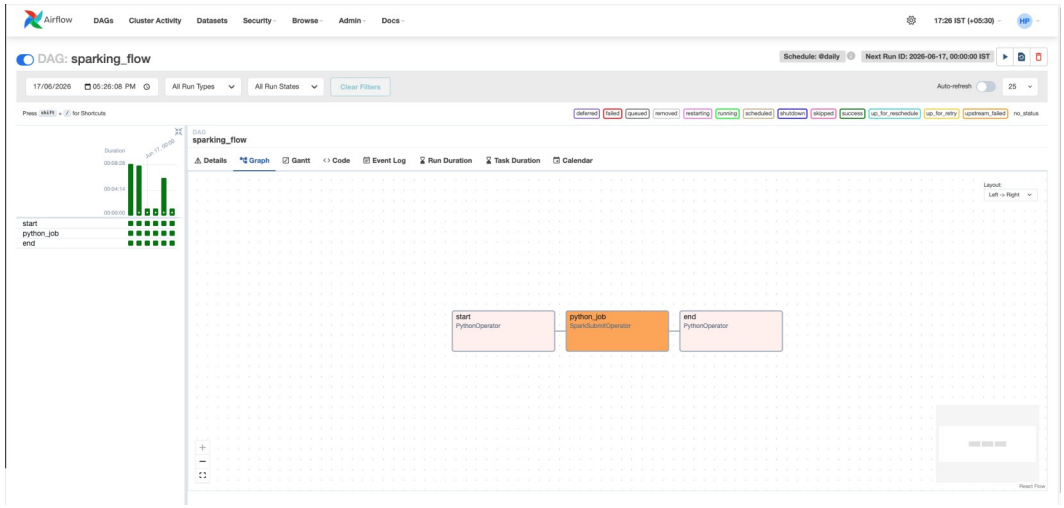


Figure 1: DAG Graph View - start (PythonOperator) → python_job (SparkSubmitOperator) → end

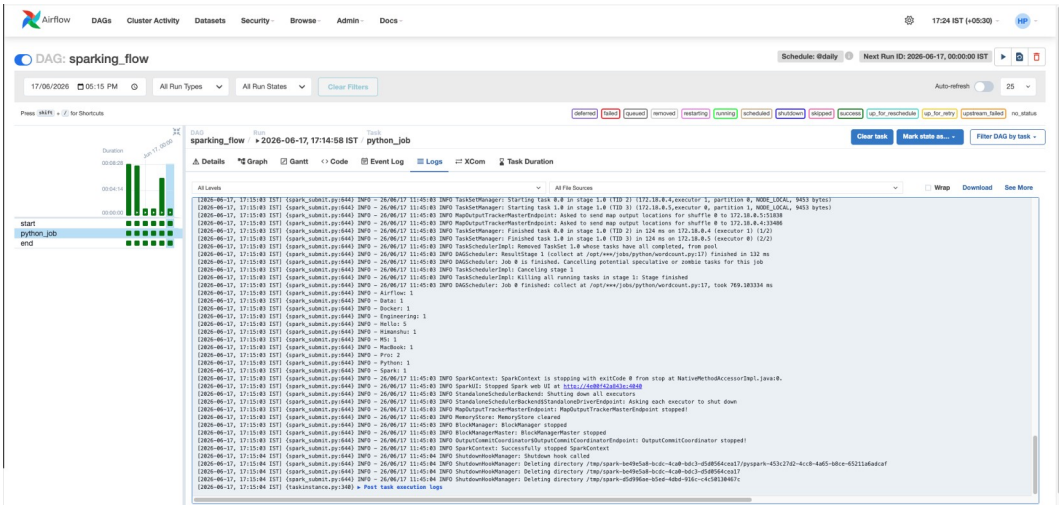


Figure 2: Successful python_job Logs - WordCountJob completed with expected output (Airflow: 1, Data: 1, Docker: 1, etc.)

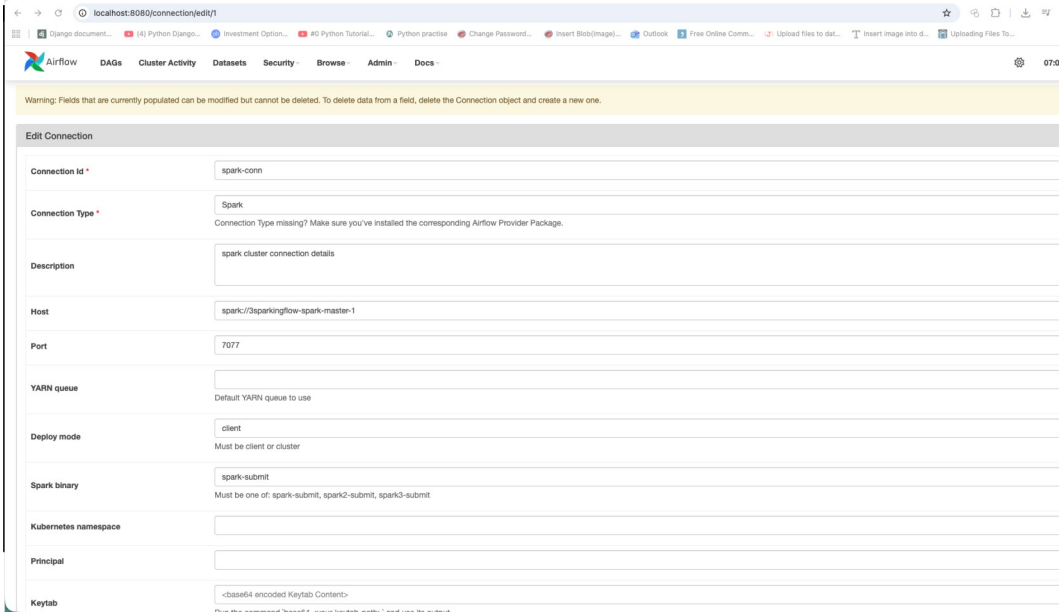
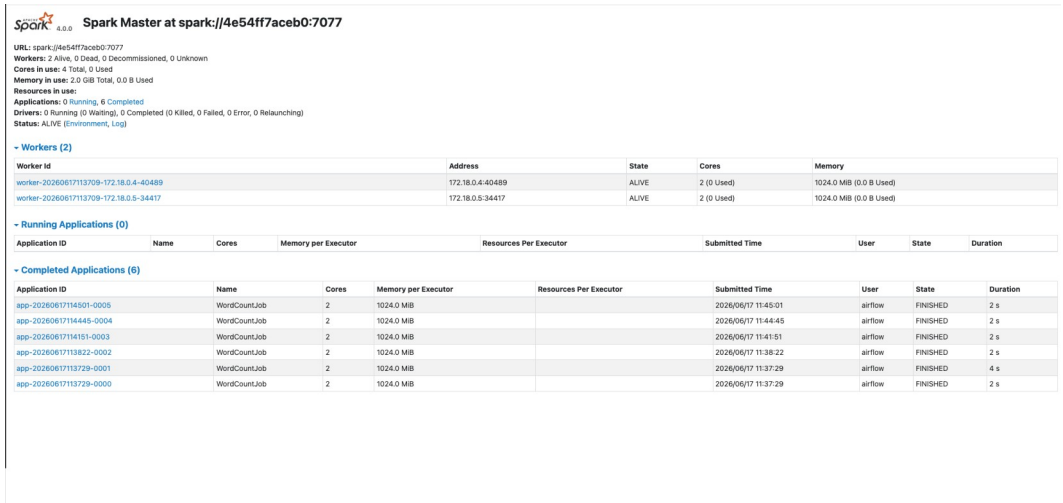


Figure 3: Spark Connection Configuration (spark-conn) - Host: spark://...spark-master-1, Port 7077, Deploy Mode: client



Spark 4.0.0 **Spark Master at spark://4e54ff7aceb0:7077**

URL: spark://4e54ff7aceb0:7077
Workers: 2 Alive, 0 Dead, 0 Decommissioned, 0 Unknown
Cores in use: 4 Total, 0 Used
Memory in use: 2.0 GiB Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 6 Completed
Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)
Status: ALIVE (Environment, Log)

Workers (2)

Worker Id	Address	State	Cores	Memory
worker-20260617113709-172.18.0.4-40489	172.18.0.4:40489	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)
worker-20260617113709-172.18.0.5-34417	172.18.0.5:34417	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)

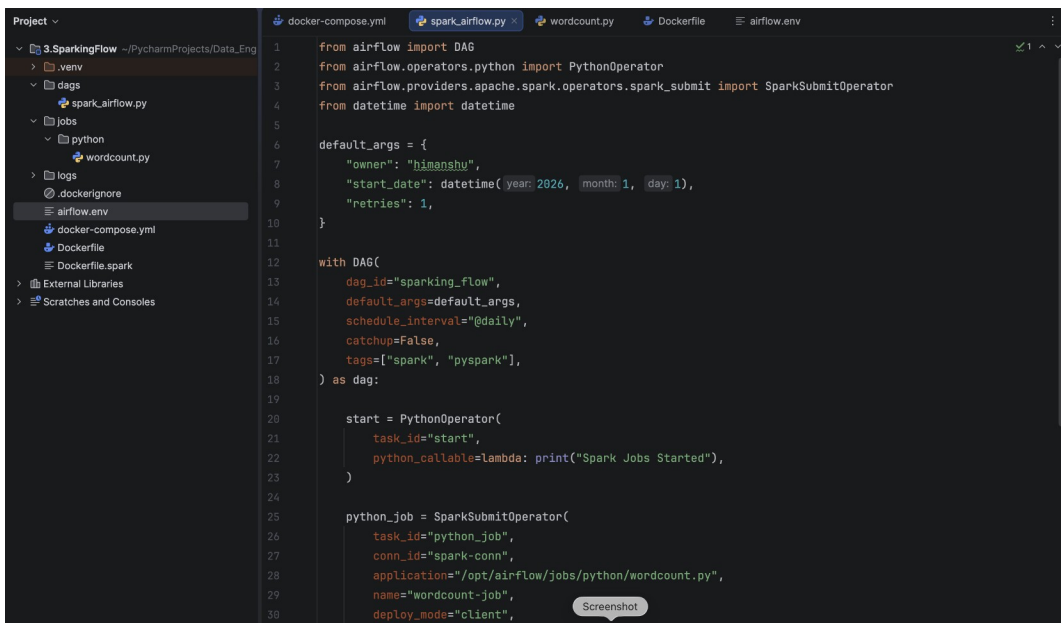
Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration

Completed Applications (6)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260617114501-0005	WordCountJob	2	1024.0 MiB		2026/06/17 11:45:01	airflow	FINISHED	2 s
app-20260617114445-0004	WordCountJob	2	1024.0 MiB		2026/06/17 11:44:45	airflow	FINISHED	2 s
app-20260617114151-0003	WordCountJob	2	1024.0 MiB		2026/06/17 11:41:51	airflow	FINISHED	2 s
app-20260617113822-0002	WordCountJob	2	1024.0 MiB		2026/06/17 11:38:22	airflow	FINISHED	2 s
app-20260617113729-0001	WordCountJob	2	1024.0 MiB		2026/06/17 11:37:29	airflow	FINISHED	4 s
app-20260617113729-0000	WordCountJob	2	1024.0 MiB		2026/06/17 11:37:29	airflow	FINISHED	2 s

Figure 4: Spark Master UI - Multiple completed WordCountJob applications from Airflow submissions



Project
3.SparkingFlow ~/PycharmProjects/Data_Eng
.venv
dags
spark_airflow.py
jobs
python
wordcount.py
logs
.dockerignore
airflow.env
docker-compose.yml
Dockerfile
Dockerfile.spark
External Libraries
Scratches and Consoles

docker-compose.yml | spark_airflow.py | wordcount.py | Dockerfile | airflow.env

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from datetime import datetime

default_args = {
    "owner": "himanshu",
    "start_date": datetime( year: 2026,  month: 1,  day: 1),
    "retries": 1,
}

with DAG(
    dag_id="sparking_flow",
    default_args=default_args,
    schedule_interval="@daily",
    catchup=False,
    tags=["spark", "pyspark"],
) as dag:

    start = PythonOperator(
        task_id="start",
        python_callable=lambda: print("Spark Jobs Started"),
    )

    python_job = SparkSubmitOperator(
        task_id="python_job",
        conn_id="spark-conn",
        application="/opt/airflow/jobs/python/wordcount.py",
        name="wordcount-job",
        deploy_mode="client",
```

Screenshot

Figure 5: Project Structure & DAG Code - spark_airflow.py with SparkSubmitOperator pointing to wordcount.py

All components verified working: DAG executed successfully, SparkSubmitOperator submitted the job to the Spark cluster, WordCountJob completed with correct output, and results visible in both Airflow logs and Spark UI.